

OBJECTIVES:

- To understand and apply the algorithm analysis techniques on searching and sorting algorithms
- To critically analyze the efficiency of graph algorithms
- To understand different algorithm design techniques
- To solve programming problems using state space tree
- To understand the concepts behind NP Completeness, Approximation algorithms and randomized algorithms.

PRACTICAL EXERCISES: 30 PERIODS

1. Using react native, build across platform application for a BMI calculator.
2. Build a cross platform application for a simple expense manager which allows entering expenses and income on each day and displays category wise weekly income and expense.
3. Develop a cross platform application to convert units from imperial system to metrics system (km to miles, kg to pounds etc.,)
4. Design and develop a cross platform application for day to day task(to-do) management.
5. Design an android application using Cordova for a user login screen with username, password, reset button and a submit button. Also, include header image and a label. Use layout managers.
6. Design and develop an android application using Apache Cordova to find and display the current location of the user.
7. Write programs using Java to create Android application having Databases
 - a) For a simple library application.
 - b) For displaying books available, books lend, book reservation. Assume that student information is available in a database which has been stored in a database server
8. The aim of the Daily Habits Tracker app is to help users track their daily habits, enabling them to cultivate positive behaviors and routines over time.

COURSE OUT COMES: At the end of this course, the students will be able to:

CO1: Develop Native applications with GUI Components.

CO2: Develop hybrid applications with basic event handling.

CO3: Implement cross-platform applications with location and data storage capabilities

CO4: Implement cross platform applications with basic GUI and event handling.

CO5: Develop web applications with cloud database access.

CO's-PO's & PSO's MAPPING

CO's	PO's												PSO's		
	1	2	3	4	5	6	7	8	9	10	11	12	1	2	3
1	2	2	1	2	3	-	-	-	1	1	2	1	2	3	3
2	2	1	3	2	2	-	-	-	3	2	2	3	3	2	1
3	2	2	2	1	2	-	-	-	1	1	1	1	1	1	2
4	1	3	1	1	3	-	-	-	1	1	3	2	1	3	1
5	1	1	3	1	3	-	-	-	1	1	2	1	3	2	1

Aim:-To create an app using react native, build across platform application for a BMI calculator.

Objective:Develop a React Native app for calculating BMI across different platforms, providing an intuitive interface for users to input their weight and height, and displaying the calculated BMI along with a brief interpretation of the result.

Theory: Body Mass Index (BMI) is a measure used to assess a person's weight status based on their height and weight. It's calculated by dividing weight (in kilograms) by the square of height (in meters). BMI falls into categories: underweight, normal weight, overweight, and obese.

Developing a BMI calculator app with React Native involves creating an interface for users to input weight and height, performing the BMI calculation, and displaying the result with interpretation. This app aims to provide a quick and accessible tool for users to monitor their weight status on different mobile platforms.

Algorithm:

Step1.Start: Begin the BMI Calculator app.

Step2.Input: Prompt the user to enter their height in centimeters. Step 3. Get Height: Capture the height input from the user.

Step4. Input: Prompt the user to enter their weight in kilograms. Step 5. Get Weight: Capture the weight input from the user.

Step6.Calculate BMI:Calculate the BMI using the formula:
$$\text{BMI} = \frac{\text{weight}}{(\text{height} \div 100)^2}$$

Step7. Determine Weight Status : Determine the weight status based on the calculated BMI. If BMI < 18.5, user is underweight; if BMI is between 18.5 and 24.9, user has normal weight; if BMI is between 25 and 29.9, user is overweight; if BMI is 30 or above, user is obese.

Step8. Display BMI Result: Show the calculated BMI along with the weight status.

Step9. Display Loading Screen:While calculating BMI, display a loading screen to indicate processing.

Step10.End: Finish the BMI Calculator app.

/Program for BMI Caluculator/

Home Sreen.js

```
import React, {useState, useEffect } from 'react';
import {Text, View, TextInput, ScrollView, Image, TouchableOpacity} from 'react-native'; import Spinner
from 'react-native-loading-spinner-overlay';
import {withExpoSnack} from 'nativewind'; import
{ styled } from 'nativewind';
importing from './assets/Group1.png';
import {useNavigation} from '@react-navigation/native';

const StyledView = styled(View); const
StyledText = styled(Text);
const StyledTextInput = styled(TextInput);

const HomeScreen = () => {
  const [height, setHeight] = useState("");
  const [weight, setWeight] = useState(""); const
  [bmi, setBMI] = useState(null);
  const [loading, setLoading] = useState(true); const

  navigation = useNavigation();

  useEffect(() => {
    const asyncOperation = async () => {
      await new Promise(resolve => setTimeout(resolve, 2000));
      setLoading(false);
    };

    asyncOperation();
  }, []);

  const calculateBMI = () => {
    setLoading(true); setTimeout(()
    => {
      const bmiValue = (weight / ((height / 100) * (height / 100))).toFixed(2); setBMI(bmiValue);
      setLoading(false);
      navigation.navigate('Result', {bmi: bmiValue});
    }, 500);
  };
};
```

```

return (
  <ScrollViewstyle={{ flex:1 }}>
    <StyledViewclassName='bg-blue-100h-screen'>
      <StyledViewclassName='m-10'>
        <StyledTextclassName='text-xlpt-0'>Welcome          </StyledText>
        <StyledTextclassName='mt-2text-3xlfont-semibold'>BMIcalculator</StyledText>
        <StyledTextclassName='pl-16text-2xlmt-5'>Checkyour<StyledTextclassName='font-
bold'>BMI</StyledText></StyledText>
        <StyledViewclassName='flexjustify-center'>
          <StyledTextclassName='mt-3text-2xl'>EnterYourHeight(cm):</StyledText>
          <StyledTextInputclassName='border-2border-blackrounded-mdpl-5'
            keyboardType="numeric"
            placeholder='EnterYourHeight' value={ height }
            onChangeText= {(text)=>setHeight(text)}
          />
        </StyledView>
        <StyledViewclassName='pb-5pt-5'>
          <StyledTextclassName='pl-0text-2xl'>EnteryourWeight(kg):</StyledText>
          <StyledTextInputclassName='border-2border-blackrounded-mdpl-5'
            keyboardType="numeric"
            placeholder='EnterYourweight' value={ weight }
            onChangeText= {(text)=>setWeight(text)}
          />
        </StyledView>

        <TouchableOpacity
          style={{ backgroundColor:'
blue',borderRadius:
10,
padding: 15, alignItems:
'center',
marginHorizontal:20,
marginTop:20
}}
          onPress={ calculateBMI }
          disabled={ loading }
        >
          <StyledTextstyle={{ color:'white',fontSize:18 }}>CalculateBMI</StyledText>
        </TouchableOpacity>

```

```

<Spinner
  visible={loading}
  textContent={'Loading...'}
  textStyle={{color:'#FFF'}}
/>
<StyledViewclassName='pl-20h-60w-60mt-14'>
  <Imagesource={img}alt='image'/>
</StyledView>
</StyledView>
</StyledView>
</ScrollView>
);
};

exportdefaultwithExpoSnack(HomeScreen);

```

ResultScreen.js

```

importReactfrom'react';
import{Button, View, Text, Image}from'react-native'; import { styled
} from 'nativewind';
importimgfrom'../assets/Group1.png';
import{ScrollView }from'react-native-gesture-handler';

constResultScreen=({route,navigation})=>{ const bmi =
  route.params?.bmi;

  const StyledView = styled(View); const
  StyledText = styled(Text);
  constStyledButton=styled(Button);

  constgetWeightStatus=(bmiValue)=>{ if
    (bmiValue< 18.5) {
      return{status:'Underweight',color:'blue'};
    }elseif(bmiValue>=18.5&&bmiValue<24.9){ return {
      status: 'Normal Weight', color: 'green'
    };
    }elseif(bmiValue>=25&&bmiValue<29.9)
    {return {status: 'Overweight', color: 'orange'
    };
    }else{

      return{status:'Obese',color:'red'};
    }
  };
};

```

```

const {status,color}=getWeightStatus(parseFloat(bmi));

return (
  <ScrollView style={{ flex:1 }}>
    <StyledView className={bg-`${color}-100h-screen`} >
      {bmi!==null&&(
        <>
          <StyledText className='pt-40 font-semibold text-xl text-center text-blue-600'> Your BMI is:
          </StyledText>
          <StyledText className={font-bold text-6xl text-center mt-5 text-`${color}-700`} >
            {bmi}
          </StyledText>
          <StyledText className={pt-5 text-center text-`${color}-800`} > Weight
            Status: {status}
          </StyledText>
        </>
      )}
    </StyledView>
    <Imagesource={img} alt='image' />
  </ScrollView>
);
};

export default ResultScreen;

```

App.js

```

import React from 'react';
import { NavigationContainer } from '@react-navigation/native'; import
import { createStackNavigator } from '@react-navigation/stack'; import
import HomeScreen from './screens/HomeScreen';
import ResultScreen from './screens/ResultScreen'; import { AppRegistry } from
'react-native';

const Stack=createStackNavigator();

const App=()=>=>
{
  return(
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home">
        <Stack.Screen name="BMICalculator" component={HomeScreen} />

```

```

        <Stack.Screenname="Result"component={ResultScreen}/>
      </Stack.Navigator>
    </NavigationContainer>
  );
};

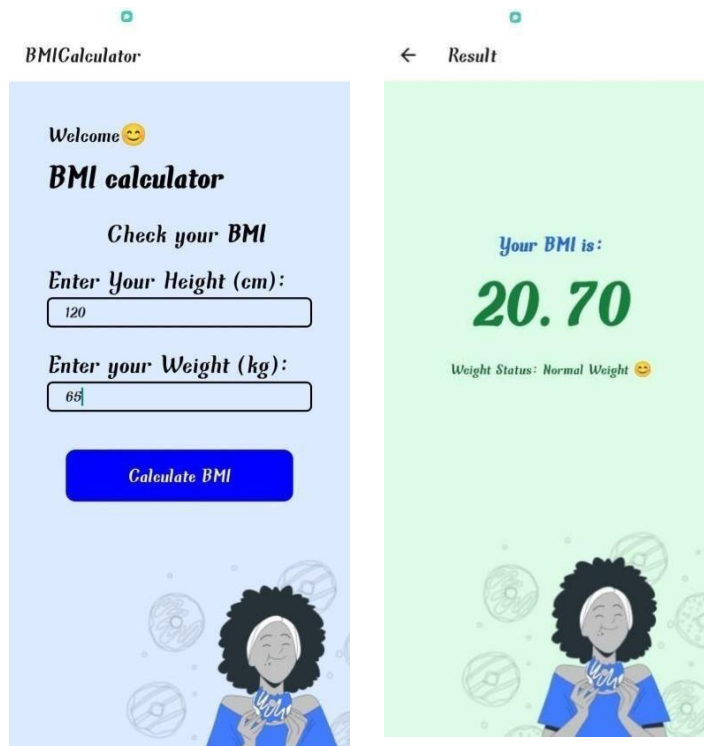
```

```

AppRegistry.registerComponent('App',()=>App); export
default App;

```

Output:



Result: Hence we have studied and executed the program for BMI Calculator.

Aim:-To Build across platform application for a simple expense manager which allows entering expenses and income on each day and displays category wise weekly income and expense.

Objective: Develop across-plat form application for managing expenses and income, enabling users to input their financial transactions daily.

Theory: A simple expense manager app enables users to track their daily expenses and income, providing a weekly summary and category-wise analysis of financial transactions to help users manage their finances effectively.

Algorithm:

Step1: Start: Begin the Daily Expenses Calculatorapp.

Step2: Load Data: On app initialization, load previously saved expense and income data from Async Storage.

Step3: Display Home Screen: Show the home screen with options to add income, add expenses, and view current balance.

Step4: Add Income: Allow the user to input their income for the selected date, along with an optional note.

Step5: Add Expense: Enable the user to input their expenses for the selected date, along with an optional note.

Step6: Show Calendar: Provide a button to display a calendar for selecting the date of income or expenses entry.

Step7: Calculate Balance: Calculate the balance by subtracting total expenses from total income, considering all transactions up to the selected date.

Step 8: Display Transactions: Show the list of income and expenses for the selected date, including the date, amount, and note (if available),with options to delete individual transactions.

Step9:Save Data: Save the updated list of income and expenses to Async Storage whenever changes occur.

Step10:End: Finish the Daily Expenses Calculator app.

/Program a simple expense manager/

HomeScreen.js

```
import React,{useState, useEffect }from'react';
import{ View,Text,TextInput,TouchableOpacity,ScrollView,Alert}from'react-native'; import {
Calendar } from 'react-native-calendars';
importModalfrom'react-native-modal'; import {
styled } from 'nativewind';
importAsyncStoragefrom'@react-native-async-storage/async-storage'; import
{ withExpoSnack } from 'nativewind';

constStyledView=styled(View); const
StyledText = styled(Text);
constStyledTextInput =styled(TextInput);

constHomeScreen=()=>>{
  const[expense,setExpense]=useState(""); const
  [income, setIncome] = useState("");
  const[isCalendarVisible,setCalendarVisible]=useState(false);
  const[selectedDate,setSelectedDate]=useState(newDate().toISOString().split('T')[0]); const
  [previousDayBalance, setPreviousDayBalance] = useState(0);
  const[expensesList,setExpensesList]=useState([]); const
  [incomeList, setIncomeList] = useState([]);
  const[expensesSummary,setExpensesSummary]=useState(null); const [note,
  setNote] = useState("");
  const[expenseNote,setExpenseNote]=useState(""); const
  [incomeNote, setIncomeNote] = useState("");

  constSTORAGE_KEY='@DailyExpenses:data';
  constMONTH_IN_MILLISECONDS=30*24*60*60*1000;

  //FunctiontoloaddatafromAsyncStorage const
  loadData = async () => {
    try{
```

```

const data = await AsyncStorage.getItem(STORAGE_KEY); if (data
!== null) {

    const parsedData = JSON.parse(data);
    const { timestamp, expensesList: storedExpenses, incomeList: storedIncome } = parsedData; if (Date.now()
- timestamp <= MONTH_IN_MILLISECONDS)
    { setExpensesList(storedExpenses || []);
      setIncomeList(storedIncome || []);
    } else { clearData();
    }
  }
} catch (error) {
  console.error('Error loading data from AsyncStorage:', error);
}
};

//Function to save data to AsyncStorage const
saveData = async () => {
  try {
    const dataToSave = JSON.stringify({
      timestamp: Date.now(), expensesList,
      incomeList,
    });
    await AsyncStorage.setItem(STORAGE_KEY, dataToSave);
  } catch (error) {
    console.error('Error saving data to AsyncStorage:', error);
  }
};

//Function to clear data from AsyncStorage const
clearData = async () => {
  try {
    await AsyncStorage.removeItem(STORAGE_KEY);
    setExpensesList([]);
    setIncomeList([]);
  } catch (error) {
    console.error('Error clearing data from AsyncStorage:', error);
  }
};

```

```

//UseuseEffecttoloaddatawhenthecomponentmounts useEffect(()
=> {
  loadData();
},[]);

//UseuseEffecttosavedatawheneverexpensesListorincomeListchanges useEffect(() => {
  saveData();
},[expensesList,incomeList]);

consthandleShowCalendar=()=>=>{
  setCalendarVisible(true);
};

consthandleAddExpense=()=>=>{
  constexpenseValue=parseFloat(expense);
  if(!isNaN(expenseValue)&&Number.isInteger(expenseValue)){
    setExpensesList([...expensesList,{ date:selectedDate,amount:expenseValue,note:expenseNote}]); setExpense("");
    setExpenseNote("");//Clearnoteafteradding
  }else {
    Alert.alert('Error','Pleaseenteravalidintegerexpenseamount.');
```

```

  }
};

consthandleAddIncome =()=>=>{
  constincomeValue=parseFloat(income);
  if(!isNaN(incomeValue)&&Number.isInteger(incomeValue)){
    setIncomeList([...incomeList,{ date:selectedDate,amount:incomeValue,note:incomeNote}]); setIncome("");
    setIncomeNote("");//Clearnoteafteradding
  }else {
    Alert.alert('Error','Pleaseenteravalidintegerincomeamount.');
```

```

  }
};

consthandleDeleteExpense=(index)=>
{ constupdatedExpenses=[...expensesList];
  updatedExpenses.splice(index, 1);
  setExpensesList(updatedExpenses);
};
```

```

consthandleDeleteIncome=(index)=>
  {constupdatedIncome=[...incomeList];
  updatedIncome.splice(index, 1);
  setIncomeList(updatedIncome);
};

constcalculateBalance=()=>{
  const totalIncome = incomeList.reduce((acc, item) => acc + parseFloat(item.amount), 0);
  consttotalExpenses=expensesList.reduce((acc,item)=>acc+parseFloat(item.amount),0); const balance
  = totalIncome - totalExpenses;

  letcolor='text-black';

  if (balance < 0)
    {color='text-red-500';
  }elseif(balance>0)
    {color='text-green-500';
  }else {
    color='text-blue-500';
  }

  return{balance,color };
};

const handleCalendarDayPress = (day) =>{
  setPreviousDayBalance(calculateBalance().balance);
  setCalendarVisible(false);
  setSelectedDate(day.dateString);//Updateselecteddate

  //Updateexpensessummarywhenadateisselected updateExpensesSummary(day.dateString);
};

constupdateExpensesSummary=(selectedDate)=>{
  //Filterexpensesfortheselectedmonth
  constselectedMonthExpenses=expensesList.filter((expense)=>expense.date.startsWith(selectedDate.slice(0, 7))
  );

  // Calculate total expenses for the selected month const
  totalExpenses =
  selectedMonthExpenses.reduce((acc,item)=>acc+
  parseFloat(item.amount),

```

```

);

setExpensesSummary({
  selectedMonth: selectedDate.slice(0, 7),
  totalExpenses: totalExpenses.toFixed(2),
  expensesList: selectedMonthExpenses,
});
};

return(
  <ScrollView>
    <StyledView className="bg-blue-300h-fullp-8">
      <StyledView className="gridgrid-cols-2">
        <TouchableOpacity onPress={handleShowCalendar} style={{
          backgroundColor: '#7C3A9F',
          padding: 10,
          borderRadius: 5,
          marginTop: 10,
          alignSelf: 'center',
        }}
      >

        <StyledText style={{ color: 'white' }}>Show Date</StyledText>
      </TouchableOpacity>
      <StyledView className="m-4">
        <StyledText className="text-lgbg-orange-100px-5py-1rounded-xl">{Selected Date:
        ${selectedDate}}</StyledText>
      </StyledView>
    </StyledView>
    <StyledText className="text-center font-semibold pb-5pt-4text-3xl">Daily Expenses
    </StyledText>

    <Modal isVisible={isCalendarVisible}>
      <Calendar onDayPress={handleCalendarDayPress}/>
    </Modal>

    <StyledView>
      <StyledText className="px-4py-2text-xl">Add Income</StyledText>
    </StyledView>
    <StyledTextInput

```

```

        className="bg-green-100px-4py-2mx-4rounded-xl" placeholder="Enter
Income"
        value={income}
        onChangeText={({text})=>setIncome(text.replace(/^[0-9]/g,""))}
        keyboardType="numeric"
    />
    <StyledViewclassName='pt-2'>
        <StyledTextInput
            className="bg-green-100px-4py-2mx-4rounded-xl"
            placeholder="Enter Note (Optional)" value={incomeNote}
            onChangeText={({text})=>setIncomeNote(text)}
        />
    </StyledView>
    <StyledViewclassName="mx-10pt-4">
        <TouchableOpacitystyle={
            {
                backgroundColor:'#27ae60',
                padding: 10,
                borderRadius:5,
                marginTop:10,
            }
        }>
            <TouchableOpacity>
                <StyledTextstyle={{color:'white'}}>Add Income</StyledText>
            </TouchableOpacity>
        </StyledView>
        <StyledView>
            <StyledTextclassName="px-4py-2text-xl">IncomeList</StyledText>
        </StyledView>
        {incomeList.map((incomeItem,index)=>(
            <StyledViewkey={index}className="flexflex-colsm:flex-rowitems-startsm:items-centerjustify-between px-4
py-2 mx-4 bg-white rounded-md my-2">
                <StyledViewclassName="flex-1">
                    <StyledTextclassName="mb-2sm:mb-0">${incomeItem.date}:
                    ${incomeItem.amount}</StyledText>
                    <StyledViewclassName="bg-yellow-100px-4py-2rounded-md">
                        <StyledTextclassName="whitespace-nowrapoverflow-hiddenoverflow-ellipsis">
                            {Note:${incomeItem.note||'-'}}
                        </StyledText>
                    </StyledView>
                </StyledView>
            </StyledView>
            <TouchableOpacityonPress={()=>handleDeleteIncome(index)}>

```

```

<StyledTextclassName="text-red-500">Delete</StyledText>
  </TouchableOpacity>
</StyledView>
)}}

<StyledView>
  <StyledTextclassName="px-4py-2text-xl">AddExpenses</StyledText>
</StyledView>
<StyledTextInput
  className="bg-red-100px-4py-2mx-4rounded-xl" placeholder="Enter
  Expenses"
  value={expense}
  onChangeText={(text)=>setExpense(text.replace(/^[0-9]/g,""))}
  keyboardType="numeric"
/>
<StyledViewclassName='pt-2'>
  <StyledTextInput
    className="bg-red-100px-4py-2mx-4rounded-xl"
    placeholder="Enter Note (Optional)" value={expenseNote}
    onChangeText={(text)=>setExpenseNote(text)}
  />
</StyledView>
<StyledViewclassName="mx-10mt-4">
  <TouchableOpacitystyle={
    {
      backgroundColor:'#3498db',
      padding: 10,
      borderRadius:5,
      marginTop:10,
    }
  }>
    <StyledTextstyle={{color:'white'}}>Add Expense</StyledText>
  </TouchableOpacity>
</StyledView>

<StyledView>
  <StyledTextclassName="px-4py-2text-xl">ExpensesList</StyledText>
</StyledView>
{expensesList.map((expenseItem,index)=>(
  <StyledViewkey={index}className="flexflex-colsm:flex-rowitems-startsm:items-centerjustify-

```



```

betweenpx-4py-2mx-4bg-whiterounded-mdmy-2">
  <StyledViewclassName="flex-1">
    <StyledTextclassName="mb-2sm:mb-0">${expenseItem.date}:
    ${expenseItem.amount}</StyledText>
    <StyledViewclassName="bg-red-100px-4py-2rounded-md">
      <StyledTextclassName="whitespace-normalbreak-words">
        {Note:${expenseItem.note}||'-'}}
      </StyledText>
    </StyledView>
  </StyledView>
  <TouchableOpacityonPress={()=>handleDeleteExpense(index)}>
    <StyledTextclassName="text-red-500">Delete</StyledText>
  </TouchableOpacity>
</StyledView>
)}}
<StyledView>
  <StyledTextclassName="px-4py-2text-xl">Balance</StyledText>
</StyledView>
<StyledViewclassName={bg-white rounded-mdmy-2p-4${calculateBalance().color}}>
  <StyledTextclassName="text-2xltext-centerfont-bold">
    {Balance:${calculateBalance().balance.toFixed(2)}}
  </StyledText>
  {previousDayBalance!==0&&(
    <StyledTextclassName="text-basetext-center">
      {PreviousDayBalance:${previousDayBalance.toFixed(2)}}
    </StyledText>
  )}
</StyledView>
<StyledView>
</StyledView>
{expensesSummary&&(
  <StyledViewclassName="bg-whiterounded-mdmy-2p-4">
    <StyledTextclassName="text-xlfont-bold">
      {ExpensesSummaryfor${expensesSummary.selectedMonth}}
    </StyledText>
    <StyledText>{TotalExpenses:${expensesSummary.totalExpenses}}</StyledText>

    <StyledViewclassName="mt-4">
      <StyledTextclassName="text-xlfont-bold">ExpensesList</StyledText>
      {expensesSummary.expensesList.map((expenseItem,index)=>(
        <StyledViewkey={inde
          x}
          className="flex-rowitems-centerjustify-between px-4py-2mx-4bg-whiterounded-mdmy-2"
        >

```

```

<StyledText>{${expenseItem.date}:${expenseItem.amount}}</StyledText>
    <StyledText>{Note:${expenseItem.note||'-'}}</StyledText>
    <TouchableOpacityonPress={()=>handleDeleteExpense(index)}>
        <StyledTextstyle={{color:'red'}}>Delete</StyledText>
    </TouchableOpacity>
</StyledView>
  )}
</StyledView>
</StyledView>
  )}
</StyledView>
</ScrollView>
);
};

```

```
exportdefaultwithExpoSnack(HomeScreen);
```

App.js

```

//App.js
importReactfrom'react';
import{NavigationContainer}from'@react-navigation/native'; import
{createStackNavigator }from'@react-navigation/stack'; import {
AppRegistry } from 'react-native';
import { withExpoSnack } from 'nativewind';
importHomeScreenfrom'./Screens/HomeScreen';

```

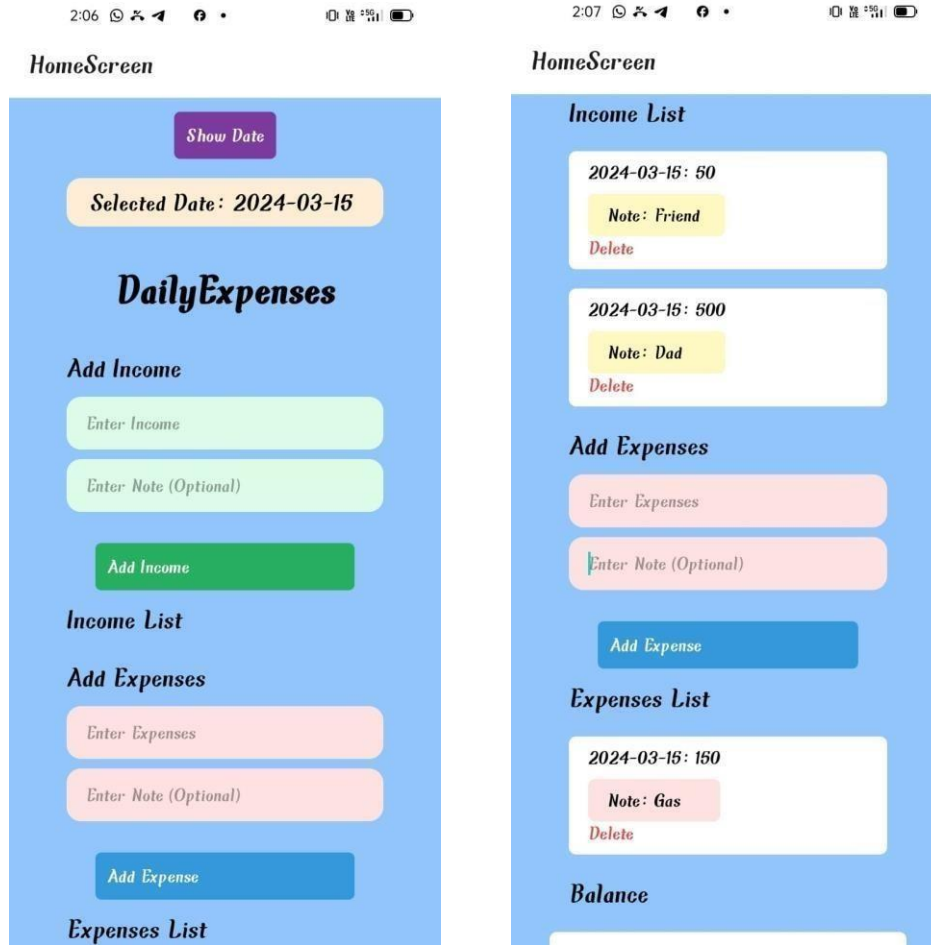
```
constStack=createStackNavigator(); const
```

```

App = () => {
  return(
    <NavigationContainer>
      <Stack.NavigatorinitialRouteName="HomeScreen">
        <Stack.Screenname="HomeScreen"component={HomeScreen}/>
      </Stack.Navigator>
    </NavigationContainer>
  );
}

```

Output:



Result: Hence we have studied and executed the program for Simple Expense Manager.

Aim:-Program to develop across platform application to convert units from imperial system to metric system (km to miles, kg to pounds etc.,)

Objective:The objective is to create across-platform application that enable users to convert units from the imperial system to the metric system and vice versa. The application will provide a user-friendly interface for converting various units such as kilometers to miles, kilograms to pounds.

Theory: The application simplifies unit conversion between the imperial and metric systems, allowing users to quickly convert measurements like kilometers to miles and kilograms to pounds.

Algorithm:

Algorithm for Setting Up and Building Unit Converter App with Cordova:

Step 1. Create Cordova Project: Use Cordova CLI to create a new Cordova project by running `cordova create Unit Converter Appcom. example. Unit converter Unit ConverterApp`. Then navigate to the project directory using `cd Unit ConverterApp`.

Step2.Add Platforms:Add platforms for Android and iOS by running `cordova platform add android` and `cordova platform add ios`.

Step 3. Design User Interface: Design the user interface in `index.html` file located in the `www` directory. Include input fields for value and selection fields for "From" and "To" units. Add a button to trigger the conversion process and a section to display the result.

Step4.Add CSSStyling: Style the user interface elements for better presentation. Create a CSS file `index.css` in the `www/css` directory and apply styles to the HTML elements.

Step 5. Implement JavaScript Functionality: Write JavaScript functionality to handle the unit conversion logic. Create a JavaScript file `index.js` in the `www/js` directory. Define a function `convert ()` to extract input values, perform unit conversion based on selected units , and display the result.

Step6. Build and Run the App: Use Cordova CLI commands to build the app for Android and iOS platforms. Run `cordova build android` and `cordova build ios` to generate the APK and IPA files respectively. Deploy the app on emulators/simulators or physical devices for testing and distribution.

/Program for cross platform application to convert units /

Set Up Your CordovaProject:

Cordova create Unit Converter App com. Example .unit converter Unit Converter App cd
Unit Converter App

Add Platforms:

```
cordova platform add android  
cordova platform add ios
```

Design the User Interface:

```
<!--index.html-->  
<!DOCTYPE html>  
<html>  
  
<head>  
  <meta charset="utf-8"/>  
  <meta name="viewport" content="width=device-width,initial-scale=1"/>  
  <title>UnitConverter</title>  
  <link rel="stylesheet" type="text/css" href="css/index.css" />  
  <script type="text/javascript" src="cordova.js"></script>  
</head>  
  
<body>  
  <div class="container">  
    <h2>UnitConverter</h2>  
    <label for="value">Value:</label>  
    <input type="number" id="value" name="value"/>  
  
    <label for="fromUnit">From:</label>  
    <select id="fromUnit">  
      <option value="km">Kilometers(km)</option>  
      <option value="mi">Miles(mi)</option>  
      <!--Add more options for other units-->  
    </select>  
  
    <label for="toUnit">To:</label>  
    <select id="toUnit">
```

```

        <optionvalue="km">Kilometers(km)</option>
        <optionvalue="mi">Miles(mi)</option>
        <!--Addmoreoptionsforotherunits-->

</select>

<buttononclick="convert()">Convert</button>

<div id="result"></div>
</div>

<scriptsrc="js/index.js"></script>
</body>

</html>

```

AddCSS:

```

body{
    font-family:Arial,sans-serif;
}

.container{
    margin:20pxauto; width:
    80%;
}

label{
    display: block;
    margin-bottom:
    5px;
}

input,select
{width:100%;
padding: 8px;
margin-bottom:10px;
}

button{
    padding:10px;
    margin-top:
    10px;
}

```

Java Script Functionality:

```
// js/index.js
function convert(){
    var value = parseFloat(document.getElementById("value").value); var
    fromUnit = document.getElementById("fromUnit").value;
    var toUnit = document.getElementById("toUnit").value;

    var result = 0;
    if (fromUnit === "km" && toUnit === "mi")
        {result = value * 0.621371;
    } else if (fromUnit === "mi" && toUnit === "km")
        {result = value * 1.60934;
    }
    // Add more conversion logic for other units

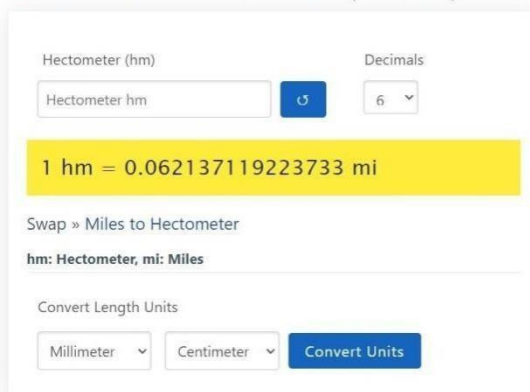
    document.getElementById("result").innerHTML = "Result:" + result.toFixed(2) + "" + toUnit;
}
```

Build and Run Your App:

Cordova build android
cordova build ios

Output:

Hectometer to Miles Conversion (hm to mi)



The screenshot shows a web interface for converting Hectometers (hm) to Miles (mi). At the top, it says "Hectometer (hm)" and "Decimals". Below this, there is a text input field containing "Hectometer hm", a blue button with a circular arrow icon, and a dropdown menu showing "6". A yellow banner displays the conversion result: "1 hm = 0.062137119223733 mi". Below the banner, there is a link "Swap » Miles to Hectometer" and a label "hm: Hectometer, mi: Miles". At the bottom, there is a section titled "Convert Length Units" with two dropdown menus: "Millimeter" and "Centimeter", and a blue button labeled "Convert Units".

Hectometer to Miles Conversions

2450 hm = 152.235942 mi	75.2 hm = 4.672711 mi
4.4 hm = 0.273403 mi	535.5 hm = 33.274427 mi
54.17 hm = 3.365968 mi	42 hm = 2.609759 mi
620 hm = 38.525014 mi	60 hm = 3.728227 mi
789 hm = 49.026187 mi	128.45 hm = 7.981513 mi
260 hm = 16.155651 mi	17.5 hm = 1.0874 mi

Kilograms to Pounds (kg to lb) conversion calculator

↕ pounds to kg (Swap units)

0kg = 0.0000lb

Accuracy 5 significant figures

Note: You can increase or decrease the accuracy of this answer by selecting the number of significant figures required from the options above the result.

Pounds = Kilograms * 2.20462

Pounds = 0 * 2.20462

Pounds = 0

Kilograms	Pounds	Kilograms	Pounds	Kilograms	Pounds	Kilograms	Pounds
0	0.0000lb	10	22.046lb	20	44.092lb	30	66.139lb
1	2.2046lb	11	24.251lb	21	46.297lb	31	68.343lb
2	4.4092lb	12	26.455lb	22	48.502lb	32	70.548lb
3	6.6139lb	13	28.660lb	23	50.706lb	33	72.752lb
4	8.8185lb	14	30.865lb	24	52.911lb	34	74.957lb
5	11.023lb	15	33.069lb	25	55.116lb	35	77.162lb
6	13.228lb	16	35.274lb	26	57.320lb	36	79.366lb

Result: Hence we have studied and executed the program for covert units from imperial system to metric system.

Aim:-Program to design and develop across platform application for day to day task (to-do) management.

Objective:To write a program to design and develop a cross platform application for day to day task (to-do) management.

Theory: The aim is to create a cross-platform application for day-to-day task management. This involves designing a user-friendly interface to add, delete, and manage tasks efficiently. The application will prioritize simplicity and intuitive functionality.

Algorithm:

1. Install Ionic CLI: Install the Ionic command-line interface globally using npm.
2. Create Ionic Project: Use the Ionic CLI to create a new Ionic project, specifying the project type as Angular.
3. Navigate to Project Directory: Change directory to the newly created project directory.
4. Design User Interface: Define the UI layout and components in the HTML file located at src/app/home/home.page.html.
5. Implement Logic: Write the TypeScript logic for the homepage component to handle adding tasks. This logic is implemented in the TypeScript file located at src/app/home/home.page.ts.
6. Build and Run: Add platforms for building (Android and iOS), build the app for each platform, and then run it on a simulator/emulator or a physical device.

/Program for TO DO list Management/

SetUp Your Ionic Project:

npm install -g @ionic/cli Create

a new Ionic project:

```
ionic start To Do List App  
blank --type=angular
```

Change directory to your
project:

```
cdToDoListApp
```

Design the User Interface:

```
<!--  
src/app/home/home.page.html -  
-->  
<ion-header>  
  <ion-toolbar>  
    <ion-title>  
      To-DoList  
    </ion-title>  
  </ion-toolbar>  
</ion-header>  
  
<ion-content>  
  <ion-list>  
    <ion-item*ngFor="let  
item of items">  
      {{ item.task }}  
    </ion-item>  
  </ion-list>  
  
  <ion-item>  
    <ion-input  
placeholder="Enter task"  
[(ngModel)]=newTask"></ion  
-input>  
    <ion-button  
expand="block"  
(click)="addItem()">Add  
Task</ion-button>  
  </ion-item>
```

Implement the Logic:

```
//src/app/home/home.page.ts
import { Component } from
 '@angular/core';

@Component({select
or: 'app-home',
templateUrl:
'home.page.html',
styleUrls:
['home.page.scss'],
})
export class HomePage
{ items: { task: string } [] =
[]; newTask: string = "";

addItem() {
  if (this.newTask.trim() !== "") {
    this.items.push({ task:
this.newTask });
    this.newTask = "";
  }
}
}
```

Build and Run Your App:

Ionic cordova platform add
android
Ionic cordova platform add
ios
Build the app:

ionic cordova build android ionic
cordova build ios

Run the app on a simulator or a
physical device:

ionic cordova emulate
android
ionic cordova emulate ios

Or run directly on a connected
device:

ionic cordova run android
ionic cordova run ios

Output:

What do you want to do today?

buy milk

☒ homework ~~3:00 pm~~

☐ football 5:00pm

☐ dinner 7:30 pm

2 tasks left.

Clear 1 completed task

Result: Hence we have studied and executed the program for Day to Day TO DO List Management

Aim:- Program to design an android application using Cordova for a user login screen with username, password, reset button and a submit button. Also, include header image and a label. Use layout managers.

Objective: Develop an Android application using Cordova to create a user-friendly login screen with input fields for username and password, along with reset and submit buttons. Additionally, include a header image and a label for enhanced visual appeal.

Theory: The aim of the program is to develop an Android application using Cordova framework, featuring a user login screen. The login screen includes input fields for username and password, along with buttons for resetting the form and submitting the login details. Additionally, the application incorporates a header image and a label for visual appeal. Layout managers are utilized to organize the elements on the screen efficiently, ensuring a user-friendly interface.

Algorithm:

1. Set Up Cordova Project: Ensure Cordova is installed, then create a new project with cordova create.
2. Add Android Platform: Add the Android platform to the project using cordova platform add android.
3. Design User Interface: Create an HTML file (index.html) for the login screen layout, including input fields for username and password, along with reset and submit buttons. Utilize CSS for styling.
4. Add JavaScript Functionality: Implement JavaScript functionality (index.js) for resetting the form and handling form submission. Capture the input values and perform necessary actions (e.g., logging credentials for demo purposes).
5. Build and Run: Build the Cordova project using cordova build android, then run it on an Android device or emulator using cordova run android.
6. Test: Test the application to ensure the login screen functions as expected, allowing users to input their credentials and submit the form.

Program for Screens:

SetUp Your Cordova Project:

First, make sure you have Cordova installed. Then create a new Cordova project:

Cordova create User Login Appcom.example.user login app User LoginApp cd
UserLoginApp

Add Android Platform:

AddAndroidplatformtoyourCordova project:

Cordova platform add android

Design the User Interface:

Create your HTML file with the login screen layout. You can use CSS for styling.

```
<!--index.html-->
<!DOCTYPEhtml>
<html>

<head>
  <metacharset="utf-8"/>
  <metaname="viewport"content="width=device-width,initial-scale=1"/>
  <title>UserLogin</title>
  <linkrel="stylesheet"type="text/css"href="css/index.css" />
</head>

<body>
  <div class="header">
    <imgsrc="header_image.png"alt="HeaderImage"/>
    <h1>Login</h1>
  </div>

  <div class="container">
    <formid="loginForm">
      <labelfor="username">Username:</label>
      <inputtype="text"id="username"name="username"/>

      <labelfor="password">Password:</label>
      <inputtype="password"id="password"name="password" />
    </form>
  </div>
</body>
</html>
```

```
        <button type="button" onclick="resetForm()">Reset</button>
        <button type="submit">Submit</button>
    </form>
</div>

<script src="js/index.js"></script>
</body>

</html>
```

Css:

```
/*css/index.css*/
body {
    font-family: Arial, sans-serif;
}

.header {
    text-align: center;
}

.container {
    margin: 20px auto; width:
    80%;
}

label {
    display: block;
    margin-bottom:
    5px;
}

input {
    width: 100%;
    padding: 8px;
    margin-bottom: 10px;
}

button {
    padding: 10px;
    margin-top:
    10px;
}
```


Java Script Functionality:

Add JavaScript file for handling form submission and resetting.

```
//js/index.js
function resetForm() {
    document.getElementById("loginForm").reset();
}

document.getElementById("loginForm").addEventListener("submit",function(event)
{ event.preventDefault(); // Prevent default form submission
varusername=document.getElementById("username").value;
varpassword=document.getElementById("password").value;

//You can addyourloginlogichere
// For demo purposes, just log the credentials
console.log("Username:"+username+",Password:"+password);
});
```

Build and Run Your App:

Build your Cordova project and run it on an Android device or emulator:

```
Cordova build android
cordova run android
```

Output:

The image shows a mobile application interface for signing in. It features a green header with the text "Sign In". Below the header are two light gray rounded rectangular input fields labeled "Username" and "Password". Under the password field is a link that says "Forgot Username / Password?". Below this is a green rounded rectangular button with the text "SIGN IN". At the bottom of the screen, there is a link that says "Don't have an account?" followed by a green button labeled "SIGN UP NOW".

Result : Hence we have studied and executed the program for user login Screen.

Aim:-Program to design and develop an android application using Apache Cordova to find and display the current location of the user.

Objective: Develop an Android application using Apache Cordova to retrieve and display the user's current location.

Theory:

Apache Cordova facilitates the creation of cross-platform mobile applications by leveraging web technologies. Utilizing Cordova's geolocation plugin, we can access the device's GPS capabilities to determine the user's latitude and longitude coordinates.

These coordinates are then displayed on the application's interface. The process involves setting up a Cordova project, designing the interface using HTML and CSS, implementing JavaScript functionality.

Algorithm:

Algorithm:LocationAppUsing Cordova

1. Setup Cordova Project: Create a Cordova project named "Location App" and navigate into it.
2. Add Android Platform: Use Cordova CLI to add Android platform support.
3. InstallGeolocation Plugin:AddtheGeolocation plugintoenablelocationservices.
4. DesignUI:Createindex.htmlfordisplayinglocation information.
5. ImplementJavaScript:WriteJavaScriptinindex.jstofetchanddisplaythe user's current location.
6. Build and Run: Build the project for Android and run it on a device or emulator.
7. Test and Debug: Test the app for accurate location retrieval and handle any errors.
8. Deployment: Consider deployment options like Google Play Store for wider distribution.

//Program for android appli//

Set Up Your Cordova Project:

Cordova create Location Appcom. example.locationappLocationApp cd
LocationApp

Add the Android platform to your Cordova project:

Cordova platform add android Install

Required Plugins:

Cordova plugin addcordova -plugin- geo location

Design the User Interface:

```
<!--index.html-->
<!DOCTYPEhtml>
<html>

<head>
  <metacharset="utf-8" />
  <metaname="viewport"content="width=device-width,initial-scale=1"/>
  <title>LocationApp</title>
  <scripttype="text/javascript"src="cordova.js"></script>
</head>

<body>
  <div id="locationDisplay">
    <h2>CurrentLocation:</h2>
    <pid="latitude">Latitude:</p>
    <pid="longitude">Longitude:</p>
  </div>

  <scriptsrc="js/index.js"></script>
</body>

</html>
```

Java Script Functionality:

Add Java Script code to retrieve the current location using the Cordova Geolocation plugin and display it on the interface.

```
//js/index.js
document.addEventListener("deviceready",onDeviceReady,false);

functiononDeviceReady()
{ navigator.geolocation.getCurrentPosition(onSuccess,
  onError);
}

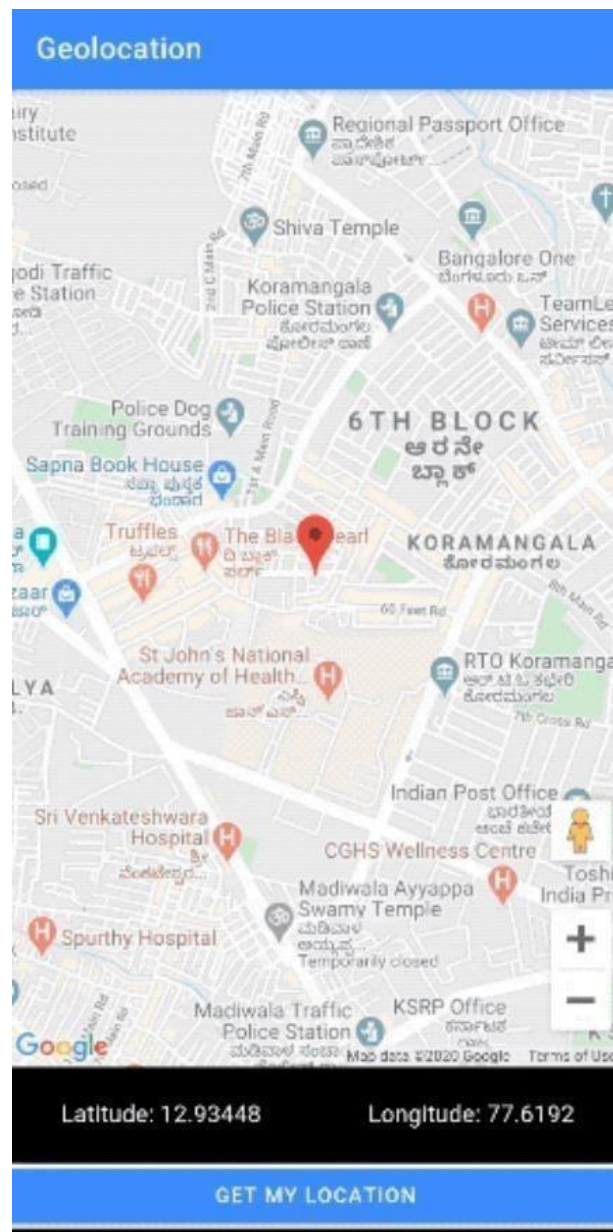
Function on Success(position){
  varlatitude=position.coords.latitude;
  var longitude = position.coords.longitude;
  document.getElementById("latitude").textContent = "Latitude: " + latitude;
  document.getElementById("longitude").textContent="Longitude:"+longitude;
}

functiononError(error){
  alert('Erroroccurred:'+error.message);
}
```

Build and RunYourApp:

```
Cordova build android
cordova run android
```

Output:



Result: Hence we have studied and executed the program to find and display the current location.

Aim:- Develop an Android library application with database support for managing book availability, loans, and reservations, utilizing a centralized database server containing student information.

Objective:- Create a user-friendly Android app enabling users to browse available books, track loaned items, and make reservations, all synchronized with a remotedatabase server housing student details.

Theory:- This Android application, built in Java, interfaces with a backend database server to facilitate library operations. Utilizing database functionalities, such as querying for books and updating loan statuses, ensures efficient management of library resources. The app's intuitive UI provides seamless navigation and displays real-time information fetched from the central database, enhancing user experience and enabling effective library management.

Algorithm:-

- Step1.Database Schema Design: Define tables for books, students, loans, and reservations.
- Step2.CreateJavaClasses: Implement classes for database operations like insertion, retrieval, and deletion.
- Step3.Database Helper Class: Develop a helper class to manage database creation and versioning.
- Step4.Perform Database Operations: Open a connection to the database, perform CRUD operations.
- Step5.Integrate with Android UI: Link database operations with user interface elements in the Android application.
- Step6.Test and Refine:Verify functionality, handle exceptions, and refine as needed for a seamless user experience.

Design Database Schema:

```
CREATETABLEbooks(  
    idINTEGERPRIMARYKEY,  
    titleTEXT,  
    author  
    TEXT,  
    availabilityINTEGER  
);
```

```

CREATETABLEstudents(
    idINTEGER PRIMARYKEY,
    nameTEXT,
    email TEXT
);

CREATETABLEloans(
    idINTEGER PRIMARYKEY,
    book_id INTEGER,
    student_idINTEGER,
    loan_date TEXT,
    return_date TEXT,
    FOREIGNKEY(book_id)REFERENCESbooks(id),
    FOREIGNKEY(student_id)REFERENCES students(id)
);

CREATETABLEreservations ( id
    INTEGER PRIMARY KEY,
    book_id INTEGER,
    student_id INTEGER,
    reservation_dateTEXT,
    FOREIGNKEY(book_id)REFERENCESbooks(id),
    FOREIGNKEY(student_id)REFERENCES students(id)
);

```

CreateJavaClassesforDatabaseOperations:

```

// DatabaseHelper.java
publicclassDatabaseHelperextendsSQLiteOpenHelper
{
    privatestaticfinalStringDATABASE_NAME=
"Library.db";
    privatestaticfinalintDATABASE_VERSION= 1;

    publicDatabaseHelper(Contextcontext)
    {super(context,DATABASE_NAME,null,
DATABASE_VERSION);
    }

    @Override
    publicvoidonCreate(SQLiteDatabasedb)
    { db.execSQL("CREATE TABLE books (id
INTEGERPRIMARYKEY,titleTEXT,authorTEXT,
availabilityINTEGER)");
}

```



```
db.execSQL("CREATETABLEstudents(id
INTEGER PRIMARY KEY, name TEXT, email TEXT)")
```

```
db.execSQL("CREATETABLEloans(idINTEGER
PRIMARY KEY, book_id INTEGER, student_id
INTEGER, loan_date TEXT, return_date TEXT, FOREIGN
KEY (book_id) REFERENCES books(id), FOREIGN KEY
(student_id) REFERENCES students(id))");
```

```
db.execSQL("CREATETABLEreservations(id
INTEGER PRIMARY KEY, book_id INTEGER,
student_id INTEGER, reservation_date TEXT,
FOREIGNKEY(book_id)REFERENCESbooks(id),
FOREIGN KEY (student_id) REFERENCES
students(id))");
}
```

```
@Override
publicvoidonUpgrade(SQLiteDatabasedb,int oldVersion,
int newVersion) {
    //Implementif needed
}
}
```

PerformDatabaseOperations:

```
// Book.java
publicclassBook
{ private int id;private
String title; private
String author; privateint
availability;

//Constructor, getters, andsetters
}
```

```
// BookRepository.java
publicclassBookRepository
{ privateSQLiteDatabase
database;privateDatabaseHelperdbHe
lper;

publicBookRepository(Contextcontext)
{ dbHelper = new
DatabaseHelper(context);}
}
```

```

publicvoidopen(){
    database=dbHelper.getWritableDatabase();
}

publicvoidclose(){ dbHelper.close();
}

publicvoidaddBook(Book book)
{ ContentValues values = new
ContentValues(); values.put("title",
book.getTitle());values.put("author",
book.getAuthor());
values.put("availability",book.getAvailability());

    database.insert("books",null,values);
}

//ImplementotherCRUDoperations
}

```

Integrate with Android UI:

```

// MainActivity.java
publicclassMainActivityextendsAppCompatActivity
{privateBookRepositorybookRepository;

@Override
protectedvoidonCreate(Bundle savedInstanceState){
    super.onCreate(savedInstanceState); setContentView(R.layout.activity_main);

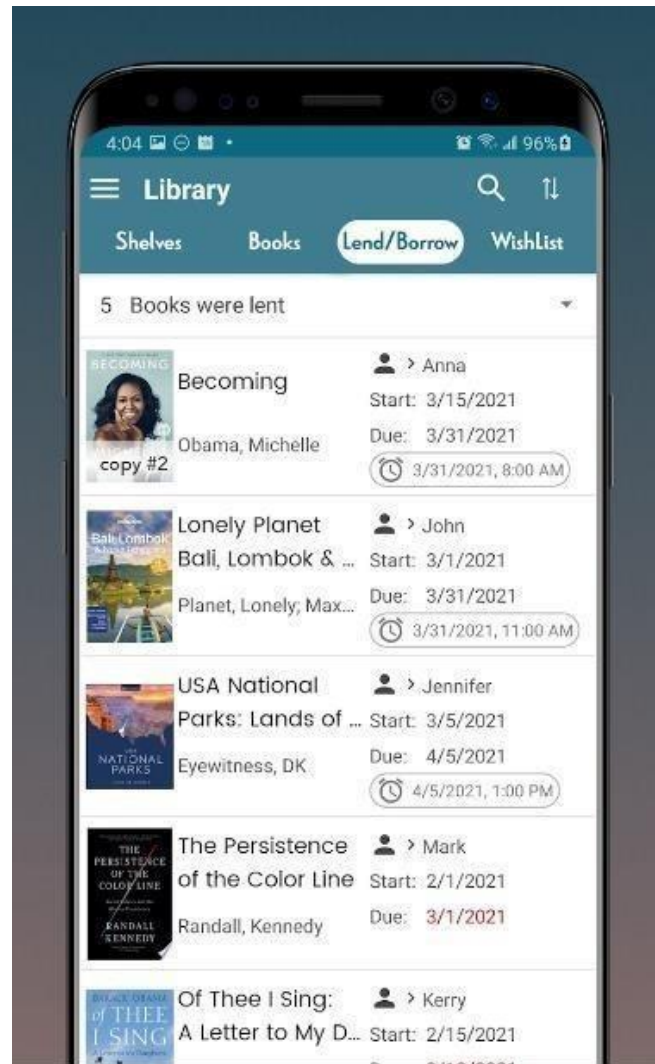
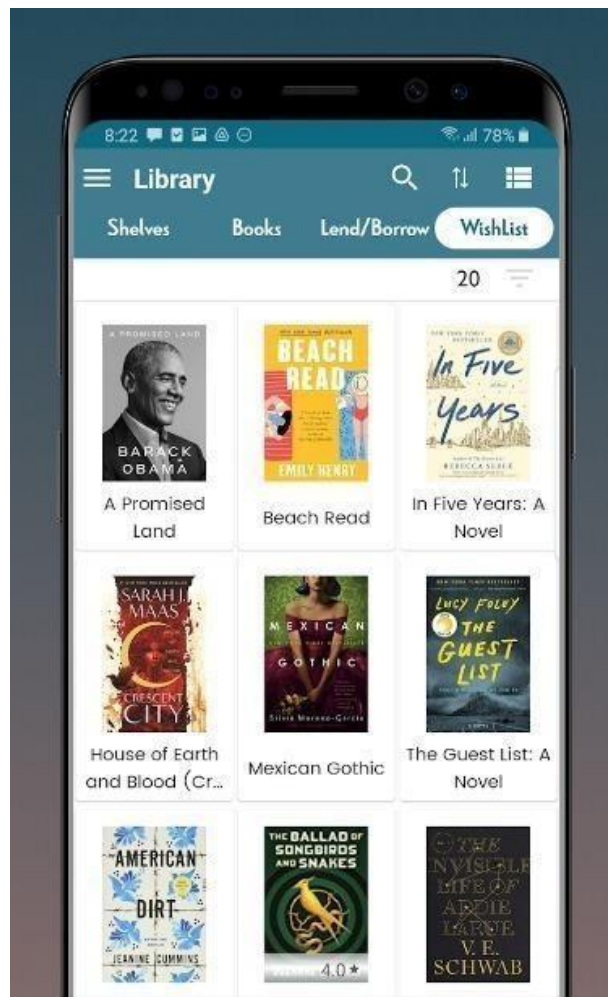
    bookRepository=newBookRepository(this);
    bookRepository.open();

    //Add a book
    Book book = new Book();
    book.setTitle("Sample Book");
    book.setAuthor("SampleAuthor");
    book.setAvailability(1);
    bookRepository.addBook(book);
}

@Override
protectedvoidonDestroy(){ super.onDestroy();
    bookRepository.close();
}
}

```

OUTPUT:



Result:-Thus the programs using Java to create Android application having Databases has been executed successfully.

Aim:-The aim of the Daily Habits Tracker app is to help users track their daily habits, enabling them to cultivate positive behaviors and routines over time.

Objective: Develop a user-friendly mobile application that allows users to track their daily habits effectively, fostering the establishment of positive behaviors and routines through consistent monitoring and feedback.

Theory:

The Daily Habits Tracker app serves as a tool to facilitate habit formation and behavior change by providing users with a structured platform to monitor their daily activities.

By logging and tracking their habits over time, users gain insight into their behavioral patterns and progress towards their goals. The app encourages consistency and accountability, empowering individuals to cultivate positive habits through regular monitoring and feedback.

Algorithm:

1. Initialize State: Create state variables to manage the list of habits (habits) and the input for adding new habits (new Habit).
2. Add Habit Functionality: Implement a function (add Habit) to add a new habit to the list when the user submits the input. Ensure that the input is not empty before adding a new habit.
3. Toggle Habit Completion: Implement a function (toggle Habit) to toggle the completion status of a habit when it is pressed. Update the completed property of the habit accordingly.
4. Render Habit List: Render the list of habits using the map function. For each habit, display its name and provide the ability to toggle its completion status by wrapping it in a Touchable Opacity.
5. Styling: Apply styles to ensure proper layout and presentation of the components. Use Style Sheet. Create to define styles for the container, input, habit container, text, and completed habits..

Program for developing app

```
import React, {useState} from 'react';
import {View, Text, TextInput, Button, TouchableOpacity, StyleSheet} from 'react-native';

const DailyHabitsTracker = () => {
  const [habits, setHabits] = useState([]);
  const [newHabit, setNewHabit] = useState("");

  const addHabit = () => {
    if (newHabit.trim() !== "") {
      setHabits([...habits, {name: newHabit, completed: false}]);
      setNewHabit("");
    }
  };

  const toggleHabit = (index) => {
    const updatedHabits = [...habits];
    updatedHabits[index].completed = !updatedHabits[index].completed;
    setHabits(updatedHabits);
  };

  return (
    <View style={styles.container}>
      <Text style={styles.title}>DailyHabitsTracker</Text>
      <View style={styles.inputContainer}>
        <TextInput style={styles.input}
          placeholder="Enter a new habit"
          value={newHabit}
          onChangeText={setNewHabit}
        />
        <Button title="Add Habit" onPress={addHabit} />
      </View>
      {habits.map((habit, index) => (
        <TouchableOpacity key={index} onPress={() => toggleHabit(index)}>
          <View style={styles.habitContainer}>
            <Text style={[styles.habitText, habit.completed && styles.completedHabit]}>
              {habit.name}
            </Text>
          </View>
        </TouchableOpacity>
      ))}
    </View>
  );
};
```

```

    ))}
  </View>
);
};

const styles =
  StyleSheet.create({ container:{
    flex: 1,
    padding: 20,alignItems:
    'center', justifyContent:
    'center',
    backgroundColor:'#fff',
  },
  title:
    { fontSize:
    24,
    fontWeight:'bold',
    marginBottom:20,
  },
  inputContainer:
    { flexDirection:
    'row',alignItems:
    'center',
    marginBottom:20,
  },
  input:
    { flex:
    1,
    borderWidth: 1,
    borderColor:'#ccc',
    borderRadius: 5,
    padding:10,
    marginRight:10,
  },
  habitContainer:
    { marginBottom:
    10,
    padding:10,
    borderWidth: 1,
    borderColor:'#ccc',
    borderRadius: 5,
  },
  habitText:
    { fontSize:
    18,},

```

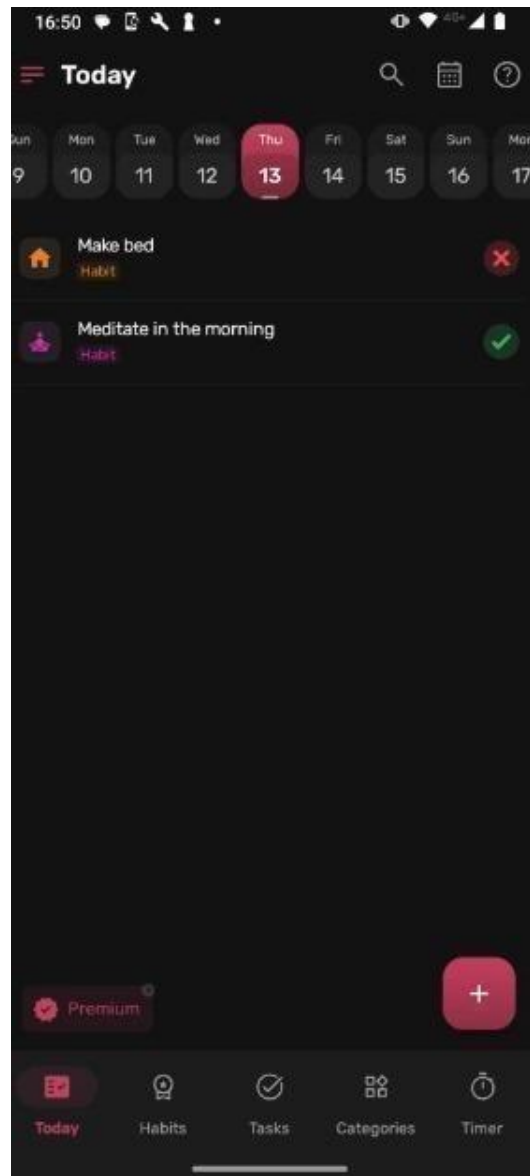
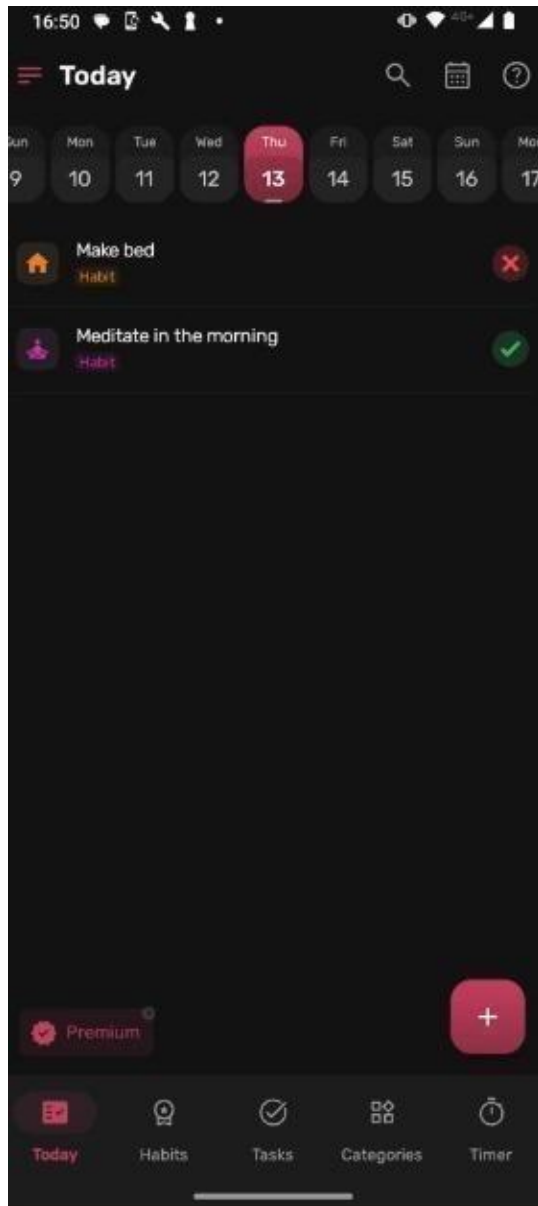
```

completedHabit:
  {textDecorationLine:'line-through',
  },
});

export default DailyHabitsTracker;

```

OUTPUT:



Result:- Thus Developing Daily Habits Tracker app is to help user s track thei r daily habits, enabling them to cultivate positive behaviors and routines over time is been executed.